Cross-Site Scripting (XSS) Prevention in PHP

1. Cross-site scripting vulnerabilities occur when web applications allow users to add custom code into a url path or onto a website that will be seen by other users. This vulnerability can be exploited to run malicious JavaScript code on a victim's browser. For example, an attacker could send an email to a victim that appears to be from a trusted bank, with a link to that bank's website. This link could have some malicious JavaScript code tagged onto the end of the url. If the bank's site is not properly protected against cross-site scripting, then that malicious code will be run in the victim's web browser when he clicks on the link.

2. Another example: It can also be performed by submitting scripts by client instead of normal text in html input forms, then admin or moderators may review these texts and execute these scripts unwillingly resulting in leakage of important information such as login session cookie, so attacker may get his hands on the admin account using this attack, unless filters and encoders are applied to the inputs in the backend side.

Prevention:

Mitigation strategies for cross-site scripting include escaping untrusted HTTP requests as well as validating and/or sanitizing user-generated content. Using encoding or escaping functions or modern web development frameworks which provides some built-in cross-site scripting protection.

Encode every value when it is written into the page, for the place where it is written. For HTML text and quoted attributes use:

1- htmlspecialchars() with the ENT_QUOTES flag and the UTF-8 charset
Example:

```
<?php
      echo htmlspecialchars($_GET['user_input'], ENT_QUOTES | ENT_SUBSTITUTE,
'UTF-8');
?>
```

2- Or The best User Input Filter is htmlpurifier library, you can read and download it from http://htmlpurifier.org/ to apply it in your project.

3- Or if you are using a PHP framework, the framework itself should have its own XSS input filtering you can use.

4- Add a Content-Security-Policy header (for example default-src 'self'; script-src 'self') as a second line of defence.

5- Don't use the '.innerHTML' property in JavaScript with untrusted data; use '.textContent' instead.

6- Don't use eval() function in javascript, needing to use eval() usually indicates a problem in your code design.

All Above Explained ideas will be Scanned in the Toolkit.

To learn more, go here:
https://www.youtube.com/watch?v=IuzU4y-UjLw
https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
https://owasp.org/Top10/2025/A05_2025-Injection/